Manuel d'utilisation

Installer, décrire un réseau, lancer une simulation, lire le résultat

Les commandes et les champs décrits ici sont lus dans le logiciel au moment de la génération, et non recopiés d'une note : une option qui change, et ce manuel change avec elle.

Version du moteur : 0.1.0 — généré le 12/11/2026 (UTC)

1. Ce que fait ce logiciel

lsim simule un réseau LoRaWAN : des capteurs qui émettent, des passerelles qui écoutent, un serveur qui commande le débit, et un canal radio qui perd ou laisse passer les trames. Il répond à des questions qu'on pose avant de déployer : combien de trames arrivent, à quelle distance, avec quelle autonomie, et à quelle vitesse le réseau réagit à une commande.

Le résultat n'est pas une estimation : c'est un calcul déterministe. Deux exécutions de la même graine donnent les mêmes compteurs, octet pour octet. C'est ce qui permet de comparer deux réglages — si le hasard s'en mêlait, une différence entre deux runs ne voudrait rien dire.

1.1 Les quatre manières de s'en servir

Mode	Pour quoi	Où com mencer
Ligne de commande	Lancer un scénario écrit à la main, produire un Parquet, un rapport ou une carte	§3
Bibliothèque Python	Enchaîner des variantes, balayer un paramètre	§4
Atelier 3D (studio web)	Poser un réseau à la souris, le régler, observer le run, faire les exercices	§6
Serveur d'intégration	Raccorder de vraies passerelles (Semtech UDP ou Basic Station) et des jointures réelles	§7

1.2 Les trois idées à connaître avant de commencer

Le scénario est validé strictement. Un champ inconnu est refusé, au lieu d'être ignoré : une faute de frappe dans un nom de champ est signalée, elle ne devient pas un réglage absent.

Rien n'est émis tant que rien n'est réglé. Un scénario sans environnement, sans région et sans profil d'énergie reste exactement celui d'avant : ses chiffres et son empreinte ne bougent pas. Ajouter une option ne change jamais rétrospectivement un résultat déjà obtenu.

Une grandeur non mesurée n'est pas publiée à zéro. Un run qui n'a pas mesuré l'énergie n'a pas de colonne d'énergie ; un run sans descente n'a pas d'attente. Une case vide se lit « pas de mesure », un zéro se lirait « mesure nulle » — et les deux ne veulent pas dire la même chose.

2. Installation

Prérequis : Python ≥ 3.10, CMake, et un compilateur C++17. mbedTLS est récupéré automatiquement par CMake — un accès à GitHub est donc nécessaire à la première configuration.

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build --parallel
pip install -e "python[data,report,test]"
```

La bibliothèque native est cherchée dans build/. Si elle est ailleurs — paquet binaire, autre répertoire de construction — on l'indique par une variable d'environnement :

```
export LSIM_LIBRARY=/chemin/vers/liblsim_core.so
```

Un paquet binaire du cœur natif (Linux) se construit avec scripts/package_native.sh : il produit une archive et son empreinte SHA-256 dans dist/. Les paquets macOS et Windows ne sont pas construits dans cet environnement.

2.1 Vérifier l'installation

La commande suivante lance un scénario d'exemple et écrit un Parquet. Si elle aboutit, le cœur natif est trouvé et le scénario est valide.

```
python -m lsim run examples/quartier_sf9.yaml -o resultats.parquet
```

3. Ligne de commande

Les six commandes ci-dessous sont celles que la version installée accepte. Leur aide est reproduite telle que le logiciel la renvoie, et non réécrite : c'est exactement ce que vous obtenez en tapant la commande.

3.1 La commande et ses sous-commandes

```
usage: lsim [-h] {run,geojson,report,serve,downlink,calibrate} ...

Simulateur LoRaWAN

positional arguments:
{run,geojson,report,serve,downlink,calibrate}
run exécuter un scénario YAML
geojson exporter les résultats en GeoJSON géoréférencé
report exécuter un scénario et écrire un rapport PDF ou Excel
serve serveur de jointure pour passerelles (Semtech UDP ou
Basic Station)
downlink mettre un downlink en file pour un dispositif
calibrate ajuster la loi log-distance sur des mesures (CSV)

options:
-h, --help show this help message and exit
```

3.2 run — exécuter un scénario

```
usage: lsim run [-h] -o OUTPUT [--energy-profile {rak4631,sx1262-stm32l072}]
scenario

positional arguments:
scenario fichier YAML du scénario

options:
-h, --help show this help message and exit
-o OUTPUT, --output OUTPUT
fichier Parquet de sortie
--energy-profile {rak4631,sx1262-stm32l072}
ajoute l'énergie par message et le courant moyen au
Parquet (rak4631 ou sx1262-stm32l072) ; prime sur
energy.profile du YAML ; absent des deux : pas de
colonnes énergie
```

3.3 report — écrire un rapport PDF ou Excel

```
usage: lsim report [-h] -o OUTPUT
[--energy-profile {rak4631,sx1262-stm32l072}]
scenario

positional arguments:
scenario fichier YAML du scénario

options:
-h, --help show this help message and exit
-o OUTPUT, --output OUTPUT
fichier de sortie (.pdf ou .xlsx)
--energy-profile {rak4631,sx1262-stm32l072}
profil énergie : rak4631 (module, MCU inclus) ou
sx1262-stm32l072 (puce + MCU séparé) ; prime sur
energy.profile du YAML ; sans l'un ni l'autre :
```

```
rak4631
```

Le format vient de l'extension du fichier de sortie : .pdf ou .xlsx. Sans option de profil, report utilise rak4631 ; run, lui, n'ajoute les colonnes d'énergie que si le profil est demandé explicitement. La différence n'est pas une inconsistance : un rapport se lit, un Parquet se traite, et un traitement qui attend des colonnes préfère qu'elles n'apparaissent que lorsqu'elles sont demandées.

3.4 geojson — exporter une carte géoréférencée

```
usage: lsim geojson [-h] -o OUTPUT [--origin ORIGIN] [--dem DEM] scenario

positional arguments:
scenario fichier YAML du scénario

options:
-h, --help show this help message and exit
-o OUTPUT, --output OUTPUT
fichier GeoJSON de sortie
--origin ORIGIN origine latitude,longitude de la passerelle (défaut :
Douala)
--dem DEM fichier JSON de MNT à écrire (scénarios avec relief
uniquement)
```

3.5 calibrate — ajuster la loi log-distance sur des mesures

```
usage: lsim calibrate [-h] [--d0 D0] [--json] measures

positional arguments:
measures CSV : distance_m,path_loss_db

options:
-h, --help show this help message and exit
--d0 D0 distance de référence d0 (m)
--json sortie JSON
```

3.6 downlink — mettre une commande en file pour un dispositif

```
usage: lsim downlink [-h] --db DB --dev-eui DEV_EUI --fport FPORT
[--payload-hex PAYLOAD_HEX] [--confirmed]
[--region {EU868,US915,AU915,AS923,KR920,IN865,RU864}]

options:
-h, --help show this help message and exit
--db DB base SQLite du registre de jointure
--dev-eui DEV_EUI DevEUI, 8 octets en hexadécimal (ordre d'affichage
TTN)
--fport FPORT FPort applicatif (1 à 223)
--payload-hex PAYLOAD_HEX
charge utile en hexadécimal
--confirmed downlink confirmé
--region {EU868,US915,AU915,AS923,KR920,IN865,RU864}
plan régional (détermine la charge maximale en file)
```

3.7 serve — serveur d'intégration des passerelles

```
usage: lsim serve [-h] [--transport {udp,basicstation}] [--host HOST]
[--port PORT] [--uri URI] [--tls-cert TLS_CERT]
[--tls-key TLS_KEY] [--hwspec HWSPEC] --db DB --keys KEYS
[--net-id NET_ID]
```

```
[--region {EU868,US915,AU915,AS923,KR920,IN865,RU864}]
[--sub-band SUB_BAND]

options:
-h, --help show this help message and exit
--transport {udp,basicstation}
udp : Semtech GWMP ; basicstation : WebSocket LNS
(défaut : udp)
--host HOST adresse d'écoute (défaut : boucle locale ; 0.0.0.0
pour le réseau)
--port PORT port (défaut : 1700 en UDP, 8887 en Basic Station)
--uri URI basicstation : adresse WebSocket publique annoncée
(ws:// ou wss://)
--tls-cert TLS_CERT basicstation : certificat PEM (wss)
--tls-key TLS_KEY basicstation : clé PEM du certificat
--hwspec HWSPEC basicstation : matériel annoncé, « sx1301/N » (défaut
: sx1301/1)
--db DB base SQLite du registre de jointure
--keys KEYS fichier JSON des clés (mode 600)
--net-id NET_ID NetID, 3 octets en hexadécimal
--region {EU868,US915,AU915,AS923,KR920,IN865,RU864}
plan régional des données et des fenêtres RX1 (défaut
: EU868)
--sub-band SUB_BAND US915/AU915 : sous-bande 1 à 8 de la passerelle
(défaut 1). Le serveur envoie un LinkADRReq à chaque
jointure et la passerelle annonce ces canaux
```

4. Utilisation en Python

La bibliothèque sert à ce qu'une ligne de commande ne sait pas faire : balayer un paramètre, comparer deux variantes, enchaîner des runs dans une seule session.

```
from lsim.export import run_scenario
from lsim.scenario import load_scenario

scenario = load_scenario("examples/quartier_sf9.yaml")
result = run_scenario(scenario, energy_profile="rak4631")
print(result.pdr, result.totals.received)
print(result.rows[0].energy_per_message_j)
```

Le hash du scénario est stable : deux exécutions avec la même graine donnent les mêmes compteurs. C'est cette propriété qui rend légitime la comparaison de deux variantes — et c'est pour elle que l'empreinte du scénario est écrite dans les métadonnées du Parquet.

4.1 Ce qu'on peut interroger directement

Chaque grandeur du modèle est exposée par une fonction, sans passer par une simulation. C'est ainsi qu'on vérifie une valeur avant de la croire :

```
from lsim import radio
from lsim.lora import RadioConfig, time_on_air_ns

radio.receiver_sensitivity_dbm(9, 125_000) # sensibilité SF9 / 125 kHz
radio.free_space_path_loss_db(1000, 868.1e6) # perte en espace libre
time_on_air_ns(RadioConfig(spreading_factor=9), 10) # temps d'antenne (ns)
```

5. Le scénario

Un scénario est un fichier YAML validé strictement : un champ inconnu est refusé. Le tableau suivant est généré depuis le contrat de validation, avec le défaut de chaque champ — il ne peut donc omettre un champ qui existe, ni en inventer un qui n'existe pas.

5.1 Champs du scénario

Champ	Type	Défaut
schema_version	Literal[1]	obligatoire
name	str	obligatoire
seed	int	0
gateway	GatewayModel	objet (voir ci-dessous)
gateways	Optional[list[GatewayModel]]	absent (facultatif)
mesh	Optional[MeshModel]	absent (facultatif)
retransmissions	Optional[RetransmissionsModel]	absent (facultatif)
propagation	PropagationModel	objet (voir ci-dessous)
environment	Optional[EnvironmentModel]	absent (facultatif)
energy	Optional[EnergyModel]	absent (facultatif)
mac	Optional[MacModel]	absent (facultatif)
groups	list[GroupModel]	obligatoire

« obligatoire » : le champ doit être présent. Les objets imbriqués sont décrits plus bas.

5.2 Les objets imbriqués

Chaque objet du scénario est décrit ci-dessous, avec le défaut de chacun de ses champs. Tous sont facultatifs : un scénario qui ne les mentionne pas garde les valeurs par défaut, et reste exactement celui d'avant.

propagation — la loi radio

Champ	Type	Défaut
frequency_hz	float	868100000.0
reference_distance_m	float	1.0
exponent	float	2.7
capture_threshold_db	float	6.0
shadowing_sigma_db	float	0.0
shadowing_seed	Optional[int]	absent (facultatif)

Les bornes sont celles de validité du modèle, pas des valeurs physiques.

gateway — la passerelle

Champ	Type	Défaut
x_m	float	0.0
y_m	float	0.0
antenna_gain_dbi	float	0.0
tx_power_dbm	Optional[float]	absent (facultatif)
name	Optional[str]	absent (facultatif)
profile	Optional[str]	absent (facultatif)

Champ	Type	Défaut
mesh	bool	False
sensitivity	Optional[SensitivityModel]	absent (facultatif)
rf	Optional[RfModel]	absent (facultatif)

Une seule passerelle : gateway. Plusieurs : gateways (même modèle, en liste).

environment — relief et obstacles

Champ	Type	Défaut
gateway_height_m	float	15.0
device_height_m	float	1.5
terrain	Optional[TerrainModel]	absent (facultatif)
obstacles	list[ObstacleModel]	PydanticUndefined

Vide, il n'existe pas : le scénario reste celui d'avant, empreinte comprise.

energy — le profil de matériel

Champ	Type	Défaut
profile	Literal['rak4631', 'sx1262-stm32l072']	obligatoire

La priorité est : option de ligne de commande, puis ce bloc, puis le défaut.

mesh — les relais

Champ	Type	Défaut
border	str	obligatoire
spreading_factor	int	9
bandwidth_khz	Literal[203, 406, 812, 1625]	203
coding_rate	int	1
frame_size	Literal['device', 'fixed']	'device'
payload_bytes	int	32
capture_db	float	6.0
max_hops	int	8
suppress_duplicates	bool	True
relay_backoff_airtimes	int	4
shadowing_sigma_db	float	0.0

Le mesh exige des passerelles multiples et un profil de relais documenté.

mac — la couche MAC

Champ	Type	Défaut
link_check_every	int	0
device_status_every	int	0
link_margin_installation_db	int	0
device_time_every	int	0
link_adr	bool	False
duty_cycle_exponent	Optional[int]	absent (facultatif)

Champ	Type	Défaut
device_channels	Optional[list[ChannelDefModel]]	absent (facultatif)
server_channel_plan	Optional[list[int]]	absent (facultatif)

Réglages de protocole ; la classe d'écoute, elle, se met par groupe.

retransmissions — uplinks confirmés

Champ	Type	Défaut
enabled	bool	True
retries	int	3
retransmit_delay_ms	int	3000
enforce_duty_cycle	bool	True
energy_profile	Optional[str]	absent (facultatif)
ack_delay_ms	int	0
ack_repetitions	int	1
downlink_payload	bool	False
session_joins_s	list[int]	PydanticUndefined
join_requests_s	list[int]	PydanticUndefined
join_retries	int	1
join_accept_rx2_fallback	bool	True
join_accept_replays	int	0
fcnt_up_start	int	0
fcnt_down_start	int	0

Bloc facultatif : sans lui, aucune trame n'est réémise.

5.3 Champs d'un groupe

Un groupe est une population de capteurs : même réglage, placements tirés selon la graine. C'est l'unité de réglage du simulateur.

Champ	Type	Défaut
name	str	obligatoire
count	Optional[int]	absent (facultatif)
placement	Union[DiskPlacement, ExplicitPlacement]	obligatoire
spreading_factor	int	7
payload_bytes	int	10
tx_power_dbm	float	14.0
period_s	float	600.0
messages	int	1
duty_cycle	float	1.0
region	Optional[str]	absent (facultatif)
channel_freq_hz	int	868100000
sub_band	Optional[int]	absent (facultatif)
start_spread_s	float	0.0
device_class	Literal['A', 'B', 'C']	'A'

Champ	Type	Défaut
downlink	Optional[DownlinkRequestsModel]	absent (facultatif)
rf	Optional[RfModel]	absent (facultatif)
rx1_dr_offset	int	0
energy	ForwardRef('Optional[EnergyModel]')	absent (facultatif)

5.4 Les plans régionaux

Régler une région sur un groupe lie le duty cycle effectif : c'est le plus restrictif entre le duty_cycle déclaré et la sous-bande du canal utilisé. Le tableau est lu dans le moteur.

Région	Canaux montants	RX2	Sous-bandes
AS923	2	923.200 MHz SF10BW125	non
AU915	72	923.300 MHz SF12BW500	oui
EU868	3	869.525 MHz SF12BW125	non
IN865	3	866.550 MHz SF10BW125	non
KR920	3	921.900 MHz SF12BW125	non
RU864	2	869.100 MHz SF12BW125	non
US915	72	923.300 MHz SF12BW500	oui

Les sources réglementaires de chaque région, avec leur statut de vérification, sont dans docs/REGULATORY_SOURCES.md et résumées dans le manuel technique §6.

5.5 Le profil d'énergie

Deux matériels sont modélisés. Le profil se choisit par le scénario (energy.profile), par l'option --energy-profile, ou par défaut ; l'ordre de priorité est : option de ligne de commande, puis YAML, puis défaut.

Profil	Matériel	Rx 125 kHz	Veille
rak4631	RAK4631 (nRF52840 + SX1262, module)	17.0 mA	2.00 µA
sx1262-stm32l072	SX1262 (datasheet) + STM32L072 (MCU)	4.6 mA	1.03 µA

Profil par défaut : rak4631. Les réserves attachées à ces valeurs — notamment le Rx du SX1262 non publié au-delà de 125 kHz — sont dans le manuel technique §4.2.

5.6 Ce que le YAML ne porte pas

Aucun frontal RF : ni rf: à la racine, ni rf: dans un groupe — le champ est refusé. Les gains d'antenne et les pertes de câble se déclarent uniquement dans l'API Python, par DeviceRf et GatewayRf. Un scénario chargé depuis YAML décrit donc une population sans frontal ; pour mesurer un gain d'antenne, il faut passer par l'API. Ce n'est pas une restriction oubliée : c'est ce qui garantit qu'un scénario écrit avant l'introduction des frontaux garde exactement ses chiffres.

6. L'atelier 3D (studio)

L'atelier est l'interface graphique : on y pose un réseau à la souris, on le règle, on lance un run, et on lit ce qui s'est passé. Il se lance en deux processus — l'API, puis le studio qui s'y adresse par un proxy.

```
python -m lsim.api_server issue-token --tokens tokens.json --owner alice
python -m lsim.api_server serve --tokens tokens.json --runs-db runs.sqlite --port 8765
cd web && npm ci && LSIM_API_URL=http://127.0.0.1:8765 npx vite --port 5173
```

Le studio est alors servi sur /studio.html. Le jeton est saisi dans la page ; il reste dans l'onglet et n'apparaît jamais dans une URL. Pour une instance locale sans saisie, l'option --auto-token <propriétaire> fait émettre un jeton au démarrage — à n'employer que sur une machine de développement, puisqu'elle désactive l'authentification.

6.1 Le tour de l'interface

Zone	Ce qu'on y fait
Barre d'outils	Nouvelle session, suppression, annuler et rétablir, enregistrer et ouvrir un chantier
Palette	Glisser un équipement sur la carte 3D pour le poser : passerelle, relais, capteur, serveur
Vues	Carte 3D (CesiumJS) et plan 2D métrique — les deux tracent les liaisons observées
Panneau d'observation	Simulation, Journal, Trame, Runs, Exercice, Obstacles
Fenêtres d'équipement	Une par équipement, trois onglets : Physique, Config, CLI. Déplaçables, pour comparer deux configurations côte à côte

6.2 L'onglet Runs : revoir et comparer

Un chantier garde ses seize derniers runs. L'onglet les liste du plus récent au plus ancien, avec la graine, la date, l'état et les grands nombres. Deux boutons par ligne :

> **Revoir** rouvre un run : ses chiffres, son plan et son journal remplacent ceux de l'écran. Le chantier ne bouge pas.
>
> **Comparer** met le run choisi face au run affiché, avec l'écart. Seules les grandeurs **mesurées par les deux** runs sont comparées : un écart calculé contre une case vide inventerait une différence.

Un run non terminé ne publie aucun chiffre : ses colonnes restent vides plutôt qu'à zéro. Un run sorti des seize n'est pas effacé, il cesse d'être rattaché — il reste consultable par son identifiant.

6.3 Le serveur de réseau

Le serveur ne se configure pas : c'est lui qui commande le débit, les canaux et les descentes. Sa fenêtre montre donc ce qu'il a **vu** du run, par équipement : trames remises sur émises, pertes hors portée puis en collision, débit commandé, canaux laissés au terminal, et les commandes envoyées quand le run en a planifié.

> Rien n'y est recopié depuis les formulaires : un champ que le run n'a pas observé reste vide, et la flèche SF7 → SF9 n'apparaît que si le serveur a commandé un changement **et** que le terminal l'a appliqué. Sinon elle recopierait le réglage de départ en laissant croire à une décision du réseau.

6.4 Obstacles, shadowing et hauteurs

Un mur se trace au rectangle sur le plan métrique. Chaque mur porte une atténuation saisie en dB, ou un matériau et une épaisseur dont le moteur calcule la perte (UIT-R P.2040-4). Rien n'est émis tant que rien n'est réglé.

Réglage	Ordre de grandeur mesuré
Mur en travers, 6 dB	à 10 km SF10 : 20 reçues sur 20 → 0 sur 20
Hauteur de passerelle	face à un mur de 12 m : 0 à 15 m, 20 sur 20 à 40 m
Shadowing σ = 8 dB	à 10 km, 40 graines : 30 restent entendues
Matériau à 868 MHz	12 des 15 hors plage documentée : perte extrapolée

Le caractère extrapolé est affiché à côté du matériau, et l'atténuation saisie reste le chemin par défaut.

6.5 Annuler et rétablir

L'historique vit côté serveur, parce que le CLI y vit aussi : annuler « region US915 » restaure la configuration de l'équipement, pas seulement la case que l'interface affichait. Un geste par opération — ajout, retrait, déplacement, ligne CLI qui change la configuration, réglages de capteur, modèle de passerelle, relief, environnement, ADR, protocole. La navigation n'en est pas un, ni un geste sans effet : quatre lignes tapées pour régler une région s'annulent d'un seul coup, et déplacer un équipement sur place n'empile rien.

Les deux boutons sont grisés **avec leur raison**, et nomment l'opération qu'ils vont annuler. Profondeur 50. Les piles ne sont pas écrites dans le fichier de runs : une session rechargée après redémarrage repart sans historique.

7. Les exercices notés

L'atelier propose des exercices corrigés automatiquement. Les critères et la solution restent côté serveur : le navigateur n'envoie jamais de résultat, il soumet l'identifiant d'une session, et le serveur la note sur l'état qu'il détient. Tricher consisterait à modifier le serveur, pas la page.

Chaque exercice a deux indices, révélés un à un ; chaque indice retire deux points, avec un plancher à zéro. Le catalogue ci-dessous est lu dans le serveur.

Exercice	Titre	Points	Indices
eu868-passerelle	Configurer une passerelle frontière sur EU868	20	2
relais-deux-sauts	Ajouter un relais à deux sauts au plus	30	2
us915-sous-bande	Choisir la sous-bande US915 et vérifier la réception	30	2
latence-frontiere	Mesurer la latence de remise au LNS	20	2
au915-sous-bande	Configurer AU915 sur la sous-bande 5	30	2
as923-sans-sous-bande	Configurer AS923 et son serveur réseau	30	2
relais-bande-large	Réduire la durée d'émission du mesh	30	2
join-flotte-etalee	Une flotte qui joint au même instant n'émet rien	40	2
adr-debit-observe	Faire changer le débit par le serveur, et le prouver	40	2

9 exercices, 270 points au total.

7.1 Ce que les exercices exigent, et pourquoi

Le plus exigeant n'est pas le plus long : c'est celui qui demande de prouver, et non de cocher. Dans « Faire changer le débit par le serveur, et le prouver », la case ADR cochée rapporte dix points ; les vingt autres ne tombent que si le run a **observé** un SF final plus bas que le SF de départ. À vingt messages la commande part mais le terminal ne l'applique pas encore : il en faut quarante, et c'est l'élève qui doit le découvrir en lisant le run.

De la même manière, « Une flotte qui joint au même instant n'émet rien » se corrige en étalant les départs : quatre capteurs qui émettent leur join-request à la même seconde sur le même canal se collisionnent, et personne n'est accepté. Le run passe alors de zéro à vingt uplinks de données — la leçon est dans le chiffre, pas dans l'énoncé.

7.2 Où se terminer

Un exercice se termine dans l'atelier, pas seulement au clavier : les réglages qu'il demande sont atteignables par les fenêtres de configuration, et non par la seule console. C'est une propriété vérifiée par les tests, exercice par exercice — un exercice qui obligerait à ouvrir la console serait un exercice qui enseigne un outil au lieu d'enseigner un réseau.

8. Raccorder de vraies passerelles

Le simulateur sait aussi se comporter en serveur d'intégration : une passerelle réelle peut y envoyer ses trames, et le serveur de jointure répond. Deux protocoles sont servis.

Protocole	Transport	État
Semtech UDP (GWMP v2)	UDP	PUSH_ACK, routes PULL_DATA, join-accept en PULL_RESP à tmst + 5 s
Basic Station	WebSocket	router-info, router_config, jreq, updf, dnmsg, dntxed, timesync

Le détail des formats, des bornes et des vecteurs officiels vérifiés est dans docs/USAGE.md §5bis et §5ter.

Le registre de jointure garde DevNonce et RJcount par dispositif, n'écrit qu'après vérification du MIC, et se persiste dans une base SQLite protégée par MIC. Un registre absent ou non rechargé remet les compteurs à zéro : c'est une fenêtre de rejeu. Le serveur doit donc recharger sa sauvegarde avant de servir des jointures — la remarque vaut pour une mise en production, pas pour un essai.

LR-FHSS n'est pas géré : les uplinks DR5/DR6 de US915 et la modulation LRFHSS sont refusés explicitement, et un test le vérifie. Mieux vaut un refus clair qu'un silence qui laisserait croire à une modulation simulée.

9. Lire un résultat sans se tromper

La plupart des erreurs d'interprétation viennent de quatre confusions. Elles sont faciles à éviter, à condition de savoir qu'elles existent.

On croit lire	En réalité
« Le PDR est de 0 % »	Le run n'a peut-être mesuré aucune trame. Une colonne vide n'est pas un zéro ; le simulateur ne publie pas de zéro là où il n'y a pas eu de mesure
« L'ADR a changé le débit »	La case dit ce qui a été demandé. Le SF final, dans les résultats du run, dit ce qui a été appliqué — et il peut être absent sans que rien n'ait échoué
« La passerelle entend à 40 km »	La bascule est le point où le bilan de liaison change de signe, pas une portée garantie : il n'y a ni fading ni probabilité
« Ce matériau atténue de 8 dB »	À 868 MHz, 12 des 15 matériaux sortent de la plage documentée : la perte est extrapolée, et l'interface le dit

9.1 Les indicateurs et ce qu'ils comptent

Indicateur	Ce qu'il compte	Ce qu'il ne compte pas
PDR	Trames reçues sur trames émises	Les trames jamais émises faute de canal disponible
Latence	Le temps d'antenne	Ni file d'attente, ni backhaul : c'est un plancher, pas une estimation de bout en bout
Occupation	Le temps où la passerelle est occupée	Les interférences venant d'un réseau tiers
Courant moyen	Le courant d'un message par période, en classe A	La capacité de la pile, et donc l'autonomie en années

9.2 Quand le résultat surprend

Trois causes reviennent, dans cet ordre. Un canal hors du plan régional : le scénario est refusé au chargement, le message le dit. Un duty cycle trop restrictif pour la période demandée : les messages s'espacent d'eux-mêmes, et le nombre de trames émises baisse sans que rien n'échoue. Une distance de référence d■ qui n'est pas celle qu'on croyait : à n = 2 elle n'a aucun effet, à n = 2,7 elle décale tout de 7 dB par décade.

Pour trancher, le plus court est de mesurer la grandeur directement plutôt que de raisonner sur le scénario : radio.receiver_sensitivity_dbm, free_space_path_loss_db et time_on_air_ns répondent en une ligne, et ce sont les mêmes fonctions que le simulateur appelle.

> Ce manuel a été généré depuis le logiciel ; la version au moment de la génération est 0.1.0. Les hypothèses de physique, leurs formules et leurs références sont dans le manuel technique.